Python and Conda Setup

Numerical Heat Conduction Assignments

MAE 623: Heat Conduction

The numerical assignments in this course will use Python for scientific computing, numerical methods, and plotting. The purpose of this guide is to take you from a computer with no Python setup to a working Python environment in which you can execute the example programs provided with the assignments.

You do not need previous Python experience to complete the introductory assignments. If you are familiar with MATLAB, many of the ideas will look familiar. Python uses variables, arrays, loops, functions, matrix operations, and plotting in much the same way.

The primary scientific Python packages used in this course will be:

- `NumPy` for numerical arrays, matrix operations, and numerical calculations;
- `Matplotlib` for plotting and saving figures;
- `SciPy` for additional scientific and numerical algorithms.

Assignments will generally be provided as Python scripts with filenames ending in `.py`. For example, a transient heat-conduction assignment may contain

```
transient_1d_explicit.py
transient_1d_implicit.py
```

These files are complete Python programs. You will typically modify physical parameters or numerical settings near the beginning of the file and then execute the script from a command window.

1. Install Anaconda or Miniconda

Python can be installed in several ways. For this course, the recommended approach is to use **Conda**. Conda manages Python itself as well as the scientific packages required by the assignments.

You may use either:

- **Anaconda Distribution**, which includes Python and many commonly used scientific packages; or
- **Miniconda**, which is a smaller installation containing Conda and Python. Additional scientific packages are installed as needed.

Either option is acceptable.

For Anaconda, download the installer from

https://www.anaconda.com/download

For Miniconda, see

https://docs.conda.io/projects/miniconda/en/latest/

On Windows, a normal “Just Me” installation using the default options is generally appropriate. You do not need to make Anaconda or Miniconda the system-wide Python installation.

After installation, open the Windows Start menu and search for

Anaconda Prompt

or, when using Miniconda,

Anaconda Prompt (Miniconda3)

This command window will be used to create the course environment and execute Python scripts.

On macOS or Linux, the normal Terminal application can be used after Conda has been installed.

2. Verify That Conda Is Working

Open the Anaconda Prompt or terminal and enter

```
conda --version
```

You should see a Conda version number.

You can also enter

```
conda info
```

If these commands execute without an error, Conda is installed correctly.

3. What Is a Conda Environment?

A Conda *environment* is an isolated Python installation used for a particular course, research project, or application.

Using a separate environment prevents packages required by one project from interfering with packages required by another project. It also makes the computational environment easier to reproduce.

For this course, create an environment named

`heat_conduction`

You only need to create this environment **once**. After it has been created, you simply activate it whenever you work on an assignment.

It is good practice to avoid installing course-specific packages directly into the Conda `base` environment.

4. Create the Course Python Environment

Open the Anaconda Prompt and enter

```
conda create -n heat_conduction python=3.12 numpy matplotlib scipy
```

Conda will display the packages that will be installed and ask something similar to

```
Proceed ([y]/n)?
```

Type

```
y
```

and press Enter.

The environment will now contain Python together with the principal scientific packages required for the numerical examples.

After installation is complete, activate the environment:

```
conda activate heat_conduction
```

The beginning of the command prompt should now contain

```
(heat_conduction)
```

For example, a Windows prompt may look similar to

```
(heat_conduction) C:\Users\YourName>
```

This indicates that commands such as `python` will use the Python installation inside the course environment.

5. Verify the Python Installation

With the `heat_conduction` environment active, enter

```
python --version
```

You should see Python 3.12 or a compatible version.

On Windows, determine which Python executable is being used with

```
where python
```

The first location listed should contain the `heat_conduction` environment.

On macOS or Linux, use

```
which python
```

You can also verify that NumPy, Matplotlib, and SciPy are available:

```
python -c "import numpy, matplotlib, scipy; print('Python environment is ready.')"
```

If everything is installed correctly, you should see

```
Python environment is ready.
```

6. Basic Command-Window Skills

The command window is simply another way to tell the computer which folder you are working in and which program you want to execute.

Only a small number of commands are needed for this course.

Command	What it does
`dir`	Lists files and folders in the current folder on Windows.
`cd foldername`	Moves into a folder.
`cd ..`	Moves up one folder level.
`cd /d "C:\path"`	Moves directly to a Windows folder, including a folder on another drive.
`mkdir Assignment1`	Creates a new folder named `Assignment1`.
`cls`	Clears the Windows command-window screen.
`python file.py`	Executes the Python program named `file.py`.
`where python`	Shows which Python installation Windows is currently using.
`conda info -envs`	Lists the Conda environments installed on the computer.

On macOS or Linux, the corresponding command for listing files is typically

```
ls
```

and the command for identifying Python is

```
which python
```

7. Create an Assignment Folder

It is useful to keep each assignment in its own folder.

For example, on Windows you might create

```
Documents\MAE623\Assignment1
```

One of the easiest ways to move to this folder from the Anaconda Prompt is to copy its location from Windows Explorer.

1. Open the assignment folder in Windows Explorer.
2. Click the Explorer address bar.
3. Press `Ctrl+C` to copy the complete folder path.
4. Return to the Anaconda Prompt.
5. Type `cd /d` followed by the copied path inside quotation marks.

For example:

```
cd /d "C:\Users\YOUR_USERNAME\Documents\MAE623\Assignment1"
```

Then enter

```
dir
```

to display the files contained in the folder.

The quotation marks are important when a folder name contains spaces.

8. Python Programs Are `.py` Files

A Python program, often called a *script*, is a plain-text file whose filename ends in

```
.py
```

For example:

```
transient_1d_explicit.py
```

You can edit Python files using many different programs. Examples include

- Visual Studio Code,
- Spyder,
- Notepad++,
- Windows Notepad,
- another plain-text programming editor.

A simple text editor is sufficient for these assignments.

If using Windows Notepad, make sure Windows is displaying file extensions. In Windows Explorer, select

View > Show > File name extensions

When saving from Notepad, select

Save as type: All Files (*.*)

and enter the complete filename, such as

```
transient_1d_explicit.py
```

Make sure Windows has not actually created

```
transient_1d_explicit.py.txt
```

9. Running a Python Script

Suppose the assignment folder contains

```
transient_1d_explicit.py
```

First activate the course environment:

```
conda activate heat_conduction
```

Next move to the folder containing the Python file:

```
cd /d "C:\Users\YOUR_USERNAME\Documents\MAE623\Assignment1"
```

Check that the file is present:

```
dir
```

Then execute the program:

```
python transient_1d_explicit.py
```

Python reads the file from top to bottom and executes the commands contained in the script.

A numerical heat-conduction program may perform the following sequence:

1. define the geometry and material properties;
2. create the spatial and temporal grids;
3. apply the initial and boundary conditions;
4. perform the numerical time integration;
5. store the calculated temperatures;
6. generate a plot;
7. save the plot as an image file.

For example, after running the explicit transient-conduction program, your assignment folder may contain

```
transient_1d_explicit.py
transient_1d_explicit_surface.png
```

The `.py` file contains the program. The `.png` file is a figure created by the program.

10. Explicit and Implicit Heat-Conduction Examples

For the introductory transient heat-conduction assignment, you may have two Python programs:

```
transient_1d_explicit.py
transient_1d_implicit.py
```

Both programs solve the same physical problem but use different numerical time-integration methods.

Run the explicit program using

```
python transient_1d_explicit.py
```

Then run the implicit program using

```
python transient_1d_implicit.py
```

The programs may generate image files such as

```
transient_1d_explicit_surface.png
transient_1d_implicit_surface.png
```

These images should appear in the same folder from which the Python programs were executed unless the program explicitly specifies another location.

For these examples, the final surface plots will typically show

- position on the x-axis;
- time on the y-axis;
- temperature on the surface or z-axis.

Comparing the results from the explicit and implicit programs is useful because both methods approximate the same governing differential equation using different numerical procedures.

11. Parameters You Will Commonly Modify

The numerical example programs are written so that the primary problem parameters appear near the beginning of the file.

For a one-dimensional transient heat-conduction problem, these may include quantities such as

```
k = 15.0          # Thermal conductivity, W/(m K)
rho = 7800.0      # Density, kg/m^3
cp = 500.0        # Specific heat, J/(kg K)

L = 0.10          # Length of the domain, m
T_initial = 20.0  # Initial temperature, deg. C
T_left = 100.0    # Left boundary temperature, deg. C
T_right = 20.0    # Right boundary temperature, deg. C

nx = 51           # Number of spatial nodes
dt = 0.10         # Time step, s
t_final = 100.0   # Total simulation time, s
```

For most assignments, you should modify these input values rather than rewriting the numerical method itself unless specifically instructed to do so.

Read the comments in the supplied Python files carefully. The example programs are intentionally commented to explain the numerical procedure.

12. Python Compared with MATLAB

If you are familiar with MATLAB, much of Python will be recognizable.

For example, the following Python commands create an array and calculate its square:

```
import numpy as np

x = np.linspace(0, 10, 101)
y = x**2
```

A simple plot can be generated with

```
import matplotlib.pyplot as plt

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("x^2")
plt.grid(True)
plt.show()
```

Some common MATLAB and Python differences are summarized below.

Task	MATLAB	Python / NumPy
First array index	1	0
Element-by-element power	`x.^2`	`x**2`
Matrix multiplication	`A*B`	`A @ B`
Element-by-element multiplication	`A.*B`	`A * B`
Array dimensions	`size(A)`	`A.shape`
Number of entries	`length(x)`	`len(x)`
Execute script	MATLAB Run command	`python script.py`

One particularly important difference is that Python uses **indentation** to identify commands that belong inside loops, functions, and conditional statements.

For example:

```
for i in range(5):
    value = i**2
    print(i, value)
```

The spaces before `value` and `print` are meaningful. Incorrect indentation can cause a Python error.

13. A Simple Python Test

Before running the heat-conduction programs, verify that Python, NumPy, and Matplotlib are working correctly.

Create a file named

```
test_python.py
```

and place the following code in the file:

```
import numpy as np
import matplotlib.pyplot as plt

# Create 11 equally spaced values between 0 and 10.
x = np.linspace(0, 10, 11)

# Calculate the square of each value.
y = x**2

# Print the arrays to the command window.
print("x =", x)
print("y =", y)

# Create a simple plot.
plt.plot(x, y, "o-")
plt.xlabel("x")
plt.ylabel("x^2")
plt.grid(True)
plt.tight_layout()

# Save a copy of the figure.
plt.savefig("test_python.png", dpi=300)
```

```
# Display the figure.
plt.show()

print("Calculation complete.")
print("Figure saved as test_python.png")
```

Run the program using

```
python test_python.py
```

You should see the numerical values printed in the command window and a graph appear on the screen.

After the program runs, the assignment folder should also contain

```
test_python.png
```

If this example works, your Python environment is ready for the numerical assignments.

14. Starting Work on Another Day

You do **not** recreate the environment each time you work on an assignment.

After restarting the computer, open the Anaconda Prompt and activate the existing environment:

```
conda activate heat_conduction
```

Then move to the assignment folder:

```
cd /d "C:\Users\YOUR_USERNAME\Documents\MAE623\Assignment1"
```

Check the files:

```
dir
```

Then execute the desired Python program:

```
python transient_1d_explicit.py
```

or

```
python transient_1d_implicit.py
```

When finished, you may close the Anaconda Prompt directly or deactivate the environment:

```
conda deactivate
```

15. Installing an Additional Python Package

If a future assignment requires a package that is not installed, first activate the course environment:

```
conda activate heat_conduction
```

Then install the package. For example:

```
conda install pandas
```

You can see which packages are installed using

```
conda list
```

Avoid installing course packages into the `base` environment. Activate `heat_conduction` first.

16. Common Problems

Python says that it cannot find the file.

For example:

```
python: can't open file 'transient_1d_explicit.py'
```

Use

```
dir
```

and verify that the Python file appears in the current folder. If it does not, use `cd` to move to the correct assignment folder.

A required Python module is missing.

For example:

```
ModuleNotFoundError: No module named 'numpy'
```

First make sure that the course environment is active:

```
conda activate heat_conduction
```

Then install the missing package:

```
conda install numpy
```

The wrong Python installation is being used.

On Windows, enter

```
where python
```

The first Python installation listed should belong to the `heat_conduction` Conda environment.

The program reports an error.

Python error messages are called *tracebacks*. Begin by reading the error message near the bottom. The final few lines usually identify the immediate problem and frequently give the line number at which the error occurred.

Do not immediately reinstall Python when a script produces an error. Most errors are caused by a filename, folder location, input value, indentation problem, or a small change to the Python code.

Quick Start Checklist

For most assignments, the complete workflow is:

1. Open **Anaconda Prompt**.

2. Activate the course environment:

```
conda activate heat_conduction
```

3. Open the assignment folder in Windows Explorer and copy its path.
4. Move to the assignment folder:

```
cd /d "PASTE PATH HERE"
```

5. Check that the Python program is present:

```
dir
```

6. Execute the program:

```
python filename.py
```

7. Read the output printed in the command window.
8. Check the assignment folder for generated files such as `.png` figures.
9. If an error occurs, read the final lines of the Python traceback before changing the program.

For the transient heat-conduction examples, the complete sequence will typically be

```
conda activate heat_conduction

cd /d "C:\Users\YOUR_USERNAME\Documents\MAE623\Assignment1"

dir

python transient_1d_explicit.py

python transient_1d_implicit.py
```

Helpful References

- Conda documentation
- NumPy documentation
- Matplotlib documentation
- SciPy documentation
- Python documentation